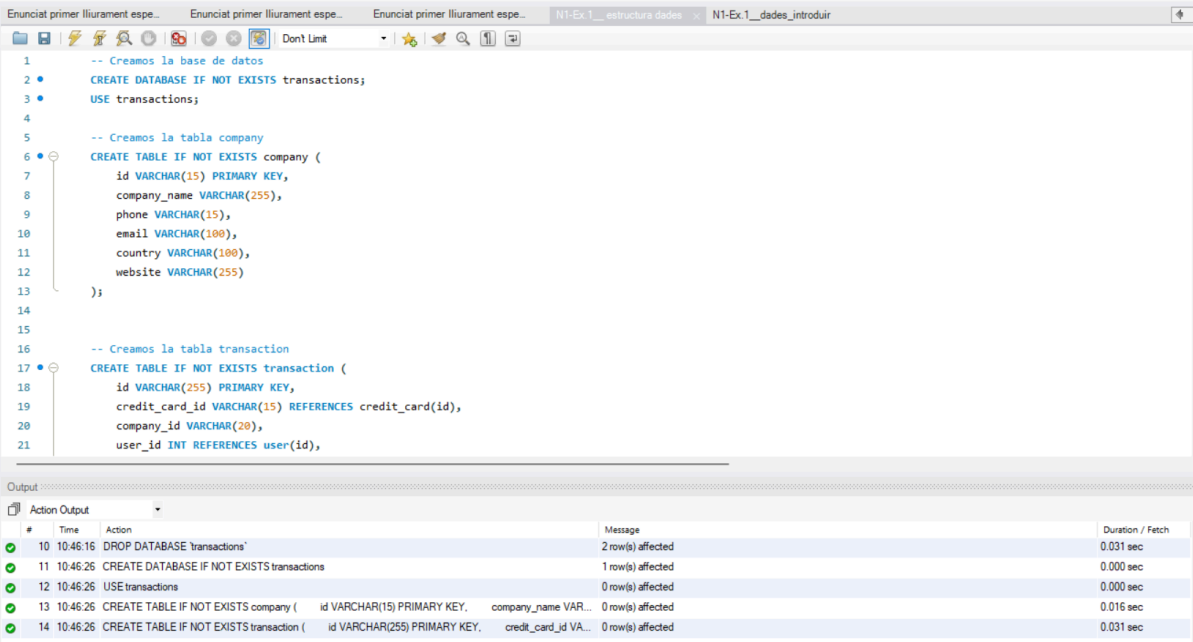


Nivell 1

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Creación de la base de datos “transactions” junto con las 2 tablas “company” y “transaction”



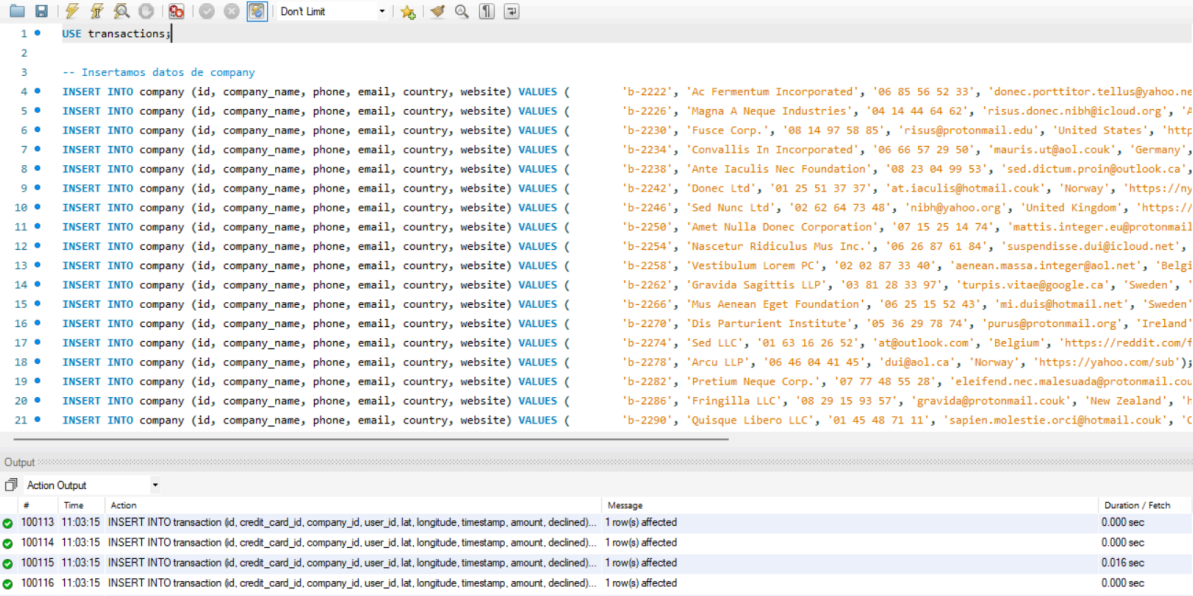
The screenshot shows a SQL IDE with a script editor and an output window. The script editor contains the following SQL code:

```
1 -- Creamos la base de datos
2 CREATE DATABASE IF NOT EXISTS transactions;
3 USE transactions;
4
5 -- Creamos la tabla company
6 CREATE TABLE IF NOT EXISTS company (
7     id VARCHAR(15) PRIMARY KEY,
8     company_name VARCHAR(255),
9     phone VARCHAR(15),
10    email VARCHAR(100),
11    country VARCHAR(100),
12    website VARCHAR(255)
13 );
14
15
16 -- Creamos la tabla transaction
17 CREATE TABLE IF NOT EXISTS transaction (
18     id VARCHAR(255) PRIMARY KEY,
19     credit_card_id VARCHAR(15) REFERENCES credit_card(id),
20     company_id VARCHAR(20),
21     user_id INT REFERENCES user(id),
```

The output window shows the execution results of the script:

#	Time	Action	Message	Duration / Fetch
10	10:46:16	DROP DATABASE transactions	2 row(s) affected	0.031 sec
11	10:46:26	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.000 sec
12	10:46:26	USE transactions	0 row(s) affected	0.000 sec
13	10:46:26	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VAR...	0 row(s) affected	0.016 sec
14	10:46:26	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VA...	0 row(s) affected	0.031 sec

Carga de los datos de las 2 tablas “company” y “transaction”



The screenshot shows a SQL IDE with a script editor and an output window. The script editor contains the following SQL code:

```
1 USE transactions;
2
3 -- Insertamos datos de company
4 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
5 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
6 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
7 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
8 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
9 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
10 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
11 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
12 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
13 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
14 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
15 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
16 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
17 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
18 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
19 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
20 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
21 INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
```

The output window shows the execution results of the script:

#	Time	Action	Message	Duration / Fetch
100113	11:03:15	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)...	1 row(s) affected	0.000 sec
100114	11:03:15	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)...	1 row(s) affected	0.000 sec
100115	11:03:15	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)...	1 row(s) affected	0.016 sec
100116	11:03:15	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)...	1 row(s) affected	0.000 sec
100117	11:03:15	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)...	1 row(s) affected	0.016 sec

Diagrama

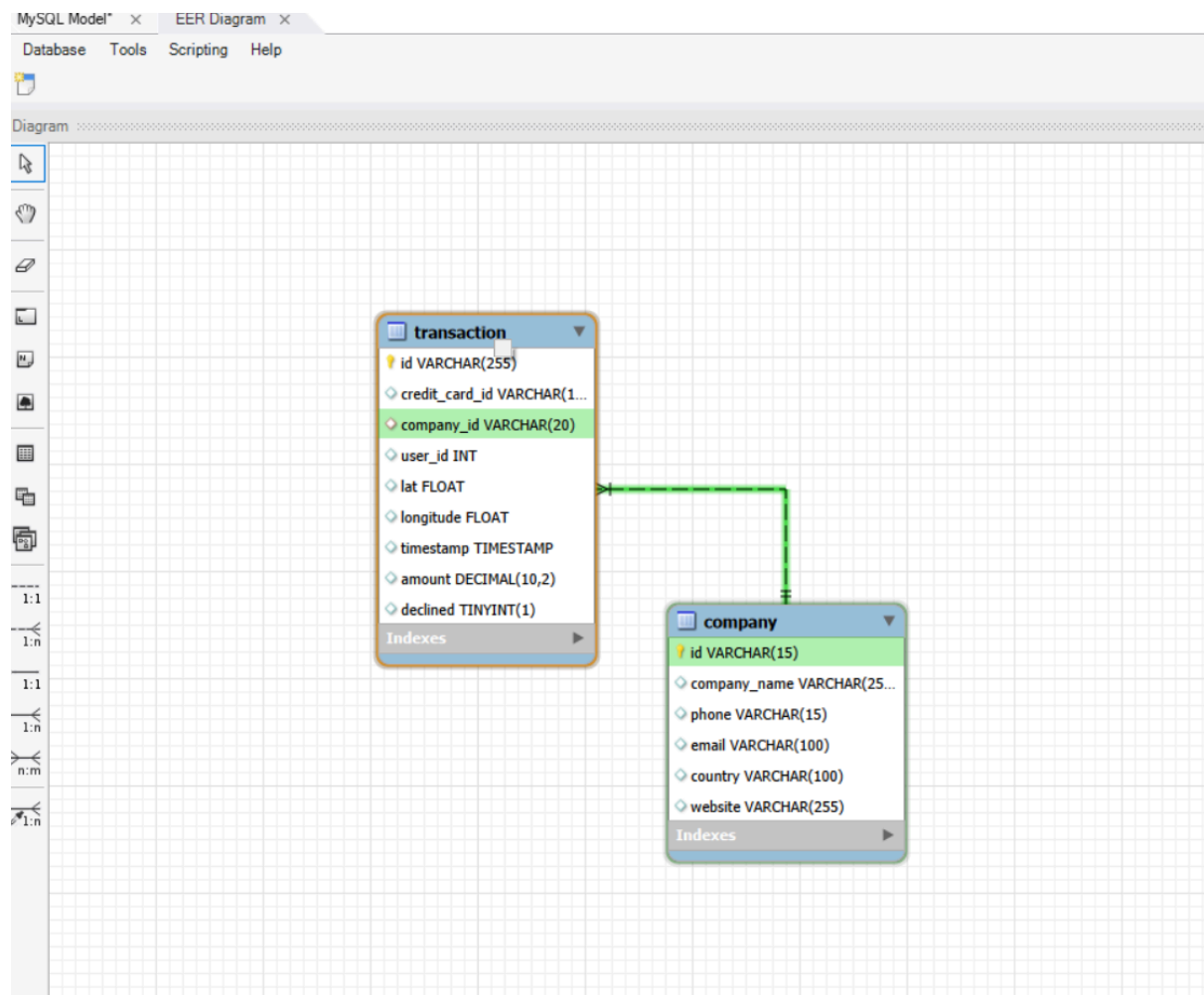
Se aprecian dos entidades “company” y “transaction” con sus correspondientes atributos.

Por ejemplo, “company” tiene una columna denominada *amount* qué es un campo con decimales. (10,2) precisa que se permiten hasta 10 dígitos en total, de los cuales 2 son decimales.

Observamos una cardinalidad de 1:N, “company” con el símbolo “one” y “transaction” con el símbolo “many”, significa que una empresa puede tener varias transacciones, pero cada transacción corresponde a una única empresa.

En “company”, la columna *id* es clave primaria, identifica de forma única a cada empresa. En “transaction”, *company_id* es clave foránea y apunta a *company.id*.

La tabla “transaction” también tiene su propia clave primaria, que es la columna *id*.



Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

Llistat dels països que estan generant vendes.

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```
37
38
39 -- Exercici 2
40 -- Utilitzant JOIN realitzaràs les següents consultes:
41
42 -- Llistat dels països que estan generant vendes.
43 • SELECT country
44 FROM transactions.company c
45 INNER JOIN transactions.transaction t ON c.id = t.company_id
46 GROUP BY country;
47
48
49 -- Des de quants països es generen les vendes
```

The results pane shows a list of countries:

country
Germany
Australia
United States
New Zealand
Norway

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
100119	11:08:36	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.265 sec
100120	12:24:21	SELECT * FROM transactions.company	100 row(s) returned	0.000 sec / 0.000 sec
100121	12:24:22	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.313 sec
100122	12:26:21	SELECT country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id = t.compan...	15 row(s) returned	0.656 sec / 0.000 sec
100123	12:28:39	SELECT country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id = t.compan...	15 row(s) returned	0.078 sec / 0.000 sec

Des de quants països es generen les vendes.

The screenshot shows a SQL IDE with a query editor and a results pane. The query is as follows:

```
41 FROM transactions.company c
42 INNER JOIN transactions.transaction t ON c.id = t.company_id
43 GROUP BY country;
44
45
46 -- Des de quants països es generen les vendes.
47 • SELECT COUNT(DISTINCT Country) AS numero_paises
48 FROM transactions.company c
49 INNER JOIN transactions.transaction t ON c.id = t.company_id;
50
51
```

The results pane shows a single row with the count of distinct countries:

numero_paises
15

The Action Output pane shows the execution log:

#	Time	Action	Message
1	15:44:32	SELECT COUNT(DISTINCT Country) AS numero_paises FROM transactions.company c INNER JOIN transacti...	1 row(s) returned

Identifica la companyia amb la mitjana més gran de vendes.

```

49 INNER JOIN transactions.transaction t ON c.id = t.company_id;
50
51
52 -- Identifica la companyia amb la mitjana més gran de vendes.
53 • SELECT ROUND(AVG(t.amount)) as total_vendas, company_name
54 FROM transactions.company c
55 INNER JOIN transactions.transaction t ON c.id = t.company_id
56 WHERE declined = 0
57 GROUP BY company_id
58 ORDER BY total_vendas
59 DESC LIMIT 1;
60

```

Result Grid

	total_vendas	company_name
▶ 285		Ac Fermentum Incorporated

Result 3 x

Output

Action Output

#	Time	Action	Message
✓ 1	15:44:32	SELECT COUNT(DISTINCT Country) AS numero_paises FROM transactions.company c INNER JOIN transacti...	1 row(s) returned
✓ 2	15:49:06	SELECT * FROM transactions.transaction	100001 row(s) returned
✗ 3	15:49:33	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER JOIN...	Error Code: 1064. You have an error in your SQL syntax; check the manu...
✓ 4	15:51:17	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER JOI...	1 row(s) returned
✓ 5	15:54:21	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER JOI...	1 row(s) returned

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

Mostra totes les transaccions realitzades per empreses d'Alemanya.

```

63
64 -- Mostra totes les transaccions realitzades per empreses d'Alemanya.
65 • SELECT *
66 FROM transactions.transaction t
67 WHERE EXISTS (
68     SELECT 1
69     FROM company c
70     WHERE c.id = t.company_id
71     AND c.country = 'Germany'
72 );
73

```

Result Grid

	id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
▶	00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
	0013C1B6-3884-4D6C-8154-E2B3FBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
	00201A11-2E62-44C4-941D-198FC8DB77F0	CdU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
	00235618-0A5C-4D49-9DCB-83A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
	005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0
	00687139-48B2-4FFA-8E73-820376F04AB4	CcS-4870	b-2222	289	51.1966	10.4669	2019-03-09 19:37:49	524.84	0
	0074F4DD-32F1-4827-8758-55896314623A	CcS-8081	b-2222	3500	39.7016	-8.50325	2016-12-26 23:06:57	491.90	0

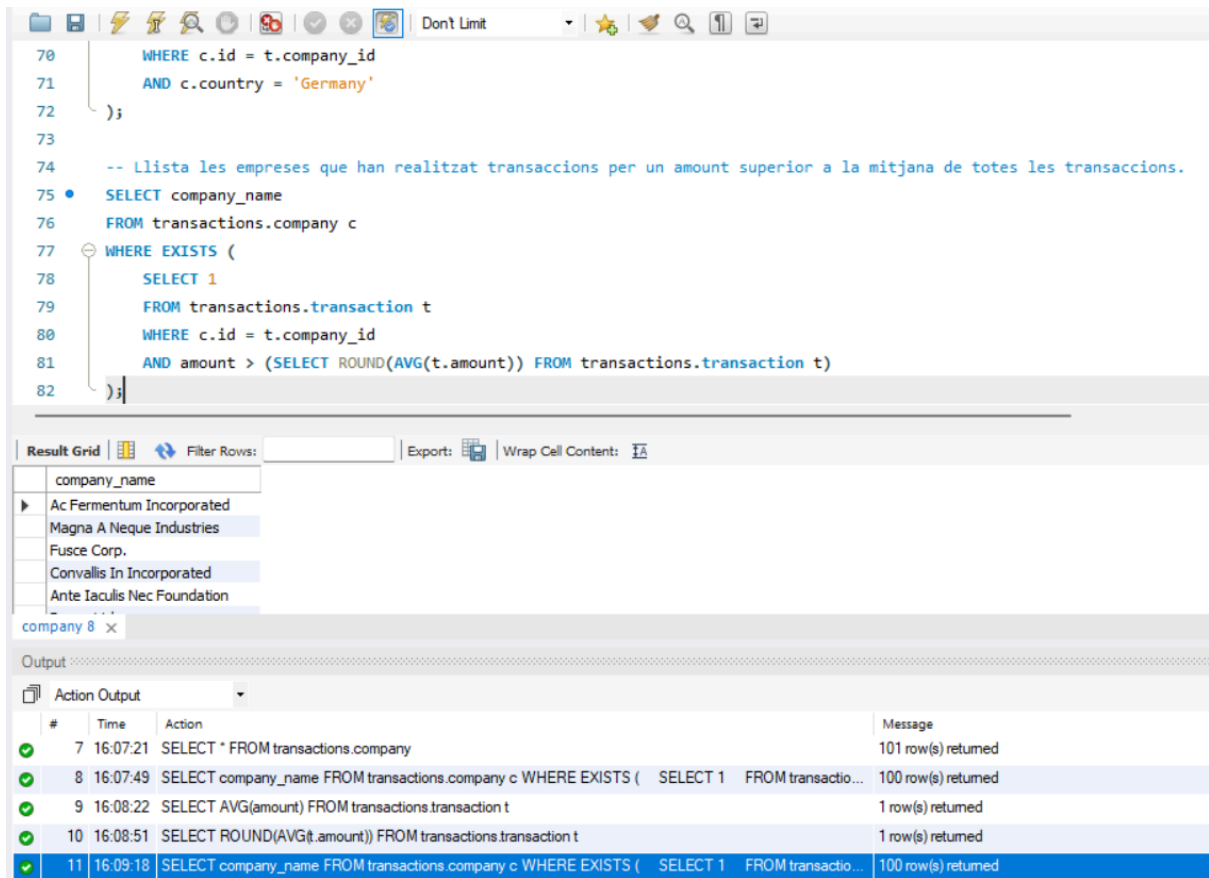
transaction 4 x

Output

Action Output

#	Time	Action	Message
✓ 2	15:49:06	SELECT * FROM transactions.transaction	100001 row(s) returned
✗ 3	15:49:33	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER JO...	Error Code: 1064. You have an error in your SQL syntax; check the m...
✓ 4	15:51:17	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER J...	1 row(s) returned
✓ 5	15:54:21	SELECT ROUND(AVG(t.amount)) as total_vendas, company_name FROM transactions.company c INNER J...	1 row(s) returned
✓ 6	15:59:39	SELECT * FROM transactions.transaction t WHERE EXISTS (SELECT 1 FROM company c WHERE ...	13291 row(s) returned

Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.



```

70 WHERE c.id = t.company_id
71 AND c.country = 'Germany'
72 );
73
74 -- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
75 • SELECT company_name
76 FROM transactions.company c
77 WHERE EXISTS (
78 SELECT 1
79 FROM transactions.transaction t
80 WHERE c.id = t.company_id
81 AND amount > (SELECT ROUND(AVG(t.amount)) FROM transactions.transaction t)
82 );

```

Result Grid

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation

company 8 x

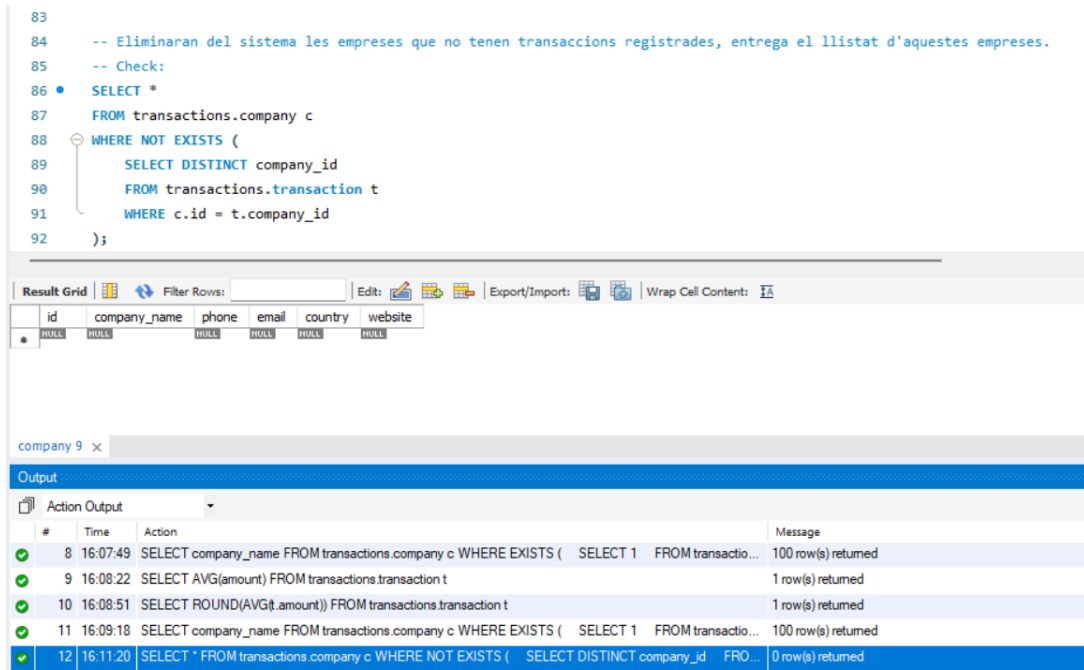
Output

#	Time	Action	Message
7	16:07:21	SELECT * FROM transactions.company	101 row(s) returned
8	16:07:49	SELECT company_name FROM transactions.company c WHERE EXISTS (SELECT 1 FROM transactio...	100 row(s) returned
9	16:08:22	SELECT AVG(amount) FROM transactions.transaction t	1 row(s) returned
10	16:08:51	SELECT ROUND(AVG(t.amount)) FROM transactions.transaction t	1 row(s) returned
11	16:09:18	SELECT company_name FROM transactions.company c WHERE EXISTS (SELECT 1 FROM transactio...	100 row(s) returned

Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.

Primero se revisa si hay empresas sin transacciones con un SELECT.

No hay ninguna con lo cual no es necesario borrar ningún dato.



```

83
84 -- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
85 -- Check:
86 • SELECT *
87 FROM transactions.company c
88 WHERE NOT EXISTS (
89 SELECT DISTINCT company_id
90 FROM transactions.transaction t
91 WHERE c.id = t.company_id
92 );

```

Result Grid

id	company_name	phone	email	country	website
*					

company 9 x

Output

#	Time	Action	Message
8	16:07:49	SELECT company_name FROM transactions.company c WHERE EXISTS (SELECT 1 FROM transactio...	100 row(s) returned
9	16:08:22	SELECT AVG(amount) FROM transactions.transaction t	1 row(s) returned
10	16:08:51	SELECT ROUND(AVG(t.amount)) FROM transactions.transaction t	1 row(s) returned
11	16:09:18	SELECT company_name FROM transactions.company c WHERE EXISTS (SELECT 1 FROM transactio...	100 row(s) returned
12	16:11:20	SELECT * FROM transactions.company c WHERE NOT EXISTS (SELECT DISTINCT company_id FRO...	0 row(s) returned

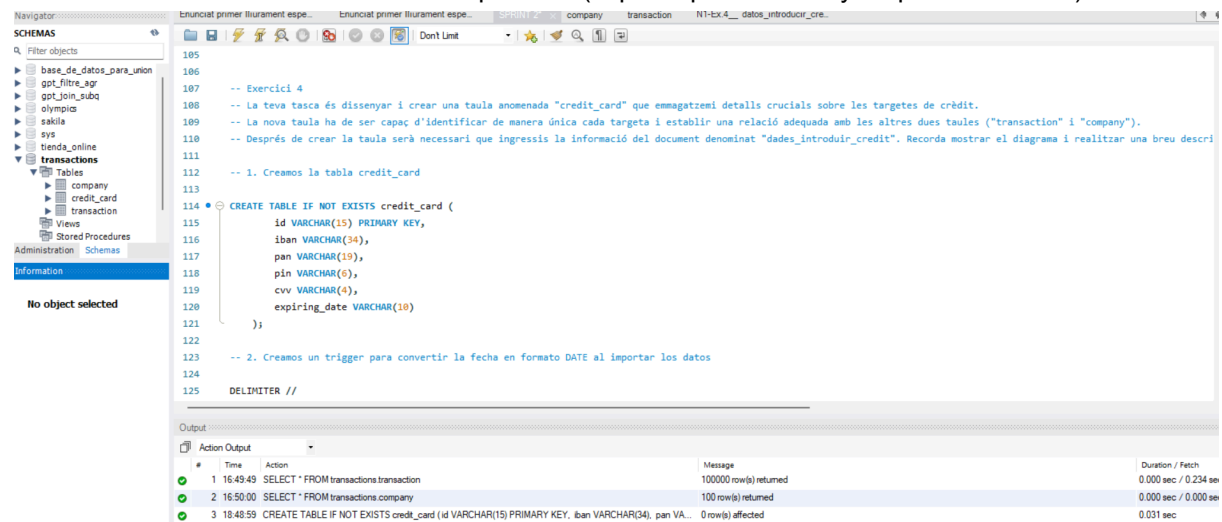
Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

1. Creación de la tabla credit_card

Dado que un iban, un pan o un pin pueden empezar por 0 y tener una longitud variable según el país emisor de la tarjeta, VARCHAR es el tipo de dato más adecuado para cubrir estos casos.

Se establece la columna id como clave primaria (implica que es única y no puede ser nula).



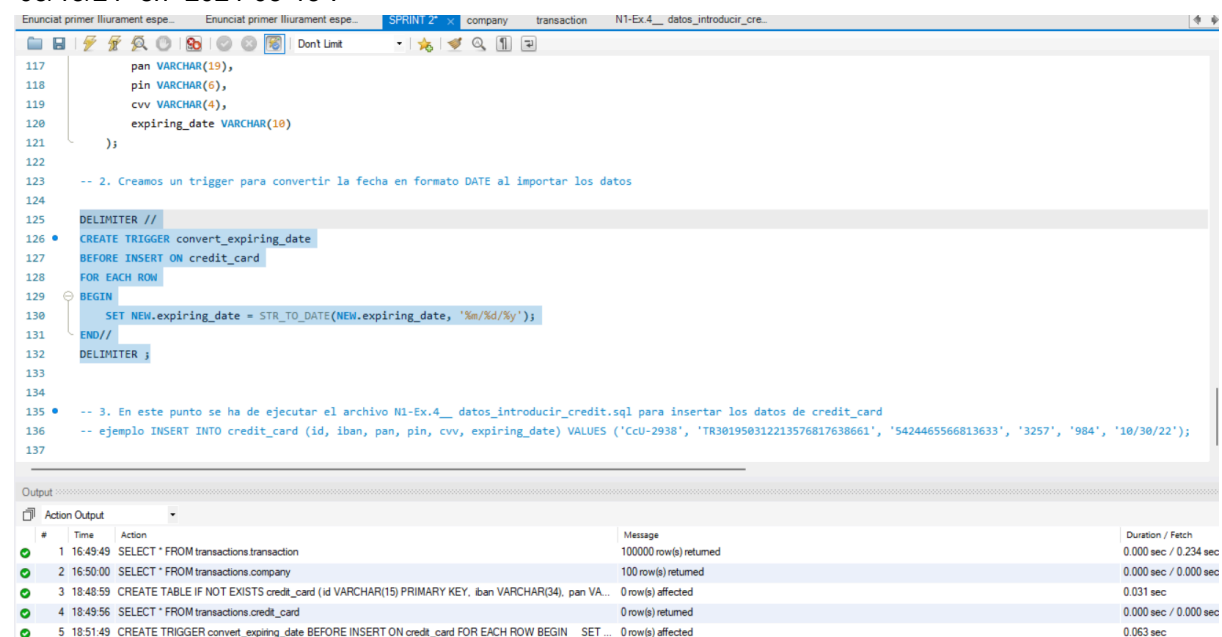
```
105
106
107 -- Exercici 4
108 -- La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit.
109 -- La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company").
110 -- Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.
111
112 -- 1. Creamos la tabla credit_card
113
114 CREATE TABLE IF NOT EXISTS credit_card (
115     id VARCHAR(15) PRIMARY KEY,
116     iban VARCHAR(34),
117     pan VARCHAR(19),
118     pin VARCHAR(6),
119     cvv VARCHAR(4),
120     expiring_date VARCHAR(10)
121 );
122
123 -- 2. Creamos un trigger para convertir la fecha en formato DATE al importar los datos
124
125 DELIMITER //
```

#	Time	Action	Message	Duration / Fetch
1	16:49:49	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.234 sec
2	16:50:00	SELECT * FROM transactions.company	100 row(s) returned	0.000 sec / 0.000 sec
3	18:48:59	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(34), pan VA...	0 row(s) affected	0.031 sec

2. Trigger

El formato de fecha del archivo a cargar "05/13/24" (mes/día/año), no corresponde al formato estándar DATE que espera MySQL (que es YYYY-MM-DD).

Para resolver esto de forma masiva para todas las filas del archivo, se utiliza un trigger que convertirá "05/13/24" en "2024-05-13".



```
117     pan VARCHAR(19),
118     pin VARCHAR(6),
119     cvv VARCHAR(4),
120     expiring_date VARCHAR(10)
121 );
122
123 -- 2. Creamos un trigger para convertir la fecha en formato DATE al importar los datos
124
125 DELIMITER //
126 CREATE TRIGGER convert_expiring_date
127 BEFORE INSERT ON credit_card
128 FOR EACH ROW
129 BEGIN
130     SET NEW.expiring_date = STR_TO_DATE(NEW.expiring_date, '%m/%d/%y');
131 END//
132 DELIMITER ;
133
134
135 -- 3. En este punto se ha de ejecutar el archivo NI-Ex.4__datos_introducir_credit.sql para insertar los datos de credit_card
136 -- ejemplo INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22');
137
```

#	Time	Action	Message	Duration / Fetch
1	16:49:49	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.234 sec
2	16:50:00	SELECT * FROM transactions.company	100 row(s) returned	0.000 sec / 0.000 sec
3	18:48:59	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(34), pan VA...	0 row(s) affected	0.031 sec
4	18:49:56	SELECT * FROM transactions.credit_card	0 row(s) returned	0.000 sec / 0.000 sec
5	18:51:49	CREATE TRIGGER convert_expiring_date BEFORE INSERT ON credit_card FOR EACH ROW BEGIN SET ...	0 row(s) affected	0.063 sec

3. Carga de los datos de la tabla credit_card

Enunciati primer lliurament espe... Enunciati primer lliurament espe... SPRINT 2 company transaction N1-Ex 4... datos_introducir_cre

Don't Limit

```
-- Insertamos datos de credit_card
1
2
3 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2938', 'TR301950312213576817638661', '5424465566813633', '3257', '984', '10/30/22');
4 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2945', 'D026854763748537475216568689', '5142423821948828', '9080', '887', '08/24/23');
5 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2952', 'B645IVQL52710525608255', '4556 453 55 5287', '4598', '438', '06/29/21');
6 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2959', 'CR7242477244335841535', '372461377349375', '3583', '667', '02/24/23');
7 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2966', 'B672LKTQ15627628377363', '448566 886747 7265', '4900', '130', '10/29/24');
8 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2973', 'PT87806228135092429456346', '544 58654 54343 384', '8760', '887', '01/30/25');
9 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2980', 'DE39241881883086277136', '402400 7145845969', '5075', '596', '07/24/22');
10 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2987', 'GE89681434837748781813', '3763 747687 76666', '2298', '797', '10/31/23');
11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-2994', 'BH62714428368066765294', '344283273252593', '7545', '595', '02/28/22');
12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3001', 'CY49087426654774581266832110', '511722 924833 2244', '9562', '867', '09/16/22');
13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3008', 'LU507216693616119230', '4485744464433884', '1856', '740', '04/05/25');
14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822', '01/31/22');
15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3022', 'GT91695162850556977423121857', '5164 1379 4842 3951', '5610', '342', '04/25/25');
16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3029', 'A262317413982441418123739746', '3429 279566 77631', '9708', '505', '09/02/23');
17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3036', 'AZ39336002925842865843941994', '3768 451556 48766', '2232', '565', '10/27/25');
18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3043', 'TN6488143310514852179535', '455676 6437463635', '5969', '196', '06/07/25');
19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3050', 'FR5167744369175836831854477', '4024007123722', '4834', '126', '10/09/23');
20 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3057', 'LU931822574697545215', '3484 621767 21237', '6805', '848', '09/14/25');
21 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('Ccu-3064', 'PS14696554449253377627273133', '3467 732741 26810', '3865', '498', '06/03/25');
```

Output

#	Time	Action	Message	Duration / Fetch
5004	18:57:21	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9577, 'XX1589140785948086...	1 row(s) affected	0.000 sec
5005	18:57:21	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9578, 'XX9915396464561105...	1 row(s) affected	0.016 sec
5006	18:57:21	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9579, 'XX2963930915871702...	1 row(s) affected	0.000 sec
5007	18:57:21	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9580, 'XX7812588898519508...	1 row(s) affected	0.000 sec
5008	18:57:21	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-9581, 'XX9156705164053881...	1 row(s) affected	0.015 sec

4. Tabla credit_card

Don't Limit

```
1 SELECT * FROM transactions.credit_card;
```

Result Grid

	id	iban	pin	cvv	expiring_date
▶	CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
	CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
	CcS-4859	XX7826930491423553609370	4983	277	2026-11-26
	CcS-4860	XX5559590368835304645299	6876	661	2027-07-27
	CcS-4861	XX2035182877195191627307	5710	398	2026-04-25
	CcS-4862	XX4774721462463645409758	4042	174	2026-11-27
	CcS-4863	XX1476829664245046207111	5969	449	2029-12-27
	CcS-4864	XX8380298893385731196159	8481	139	2026-02-28
	CcS-4865	XX7085078596101025280599	7847	903	2028-11-25
	CcS-4866	XX4792859188206596406839	9271	961	2025-02-28
	CcS-4867	XX6038298816319374853717	4820	862	2025-11-30
	CcS-4868	XX3929101617300533068044	6860	549	2025-08-25
	CcS-4869	XX1566712096111531886465	4775	448	2025-09-29

credit_card 1 x

Output

#	Time	Action	Message
9	16:08:22	SELECT AVG(amount) FROM transactions.transaction t	1 row(s) returned
10	16:08:51	SELECT ROUND(AVG(t.amount)) FROM transactions.transaction t	1 row(s) returned
11	16:09:18	SELECT company_name FROM transactions.company c WHERE EXISTS (SELECT 1 FROM transactio...	100 row(s) returned
12	16:11:20	SELECT * FROM transactions.company c WHERE NOT EXISTS (SELECT DISTINCT company_id FRO...	0 row(s) returned
13	16:20:58	SELECT * FROM transactions.credit_card	5001 row(s) returned

5. Se establece la relación de 1:n con la tabla transaction definiendo que transaction.credit_card_id es clave foranea y apunta a credit_card.id :

```

1 ALTER TABLE transaction
2 ADD CONSTRAINT fk_credit_card
3 FOREIGN KEY (credit_card_id)
4 REFERENCES credit_card(id);

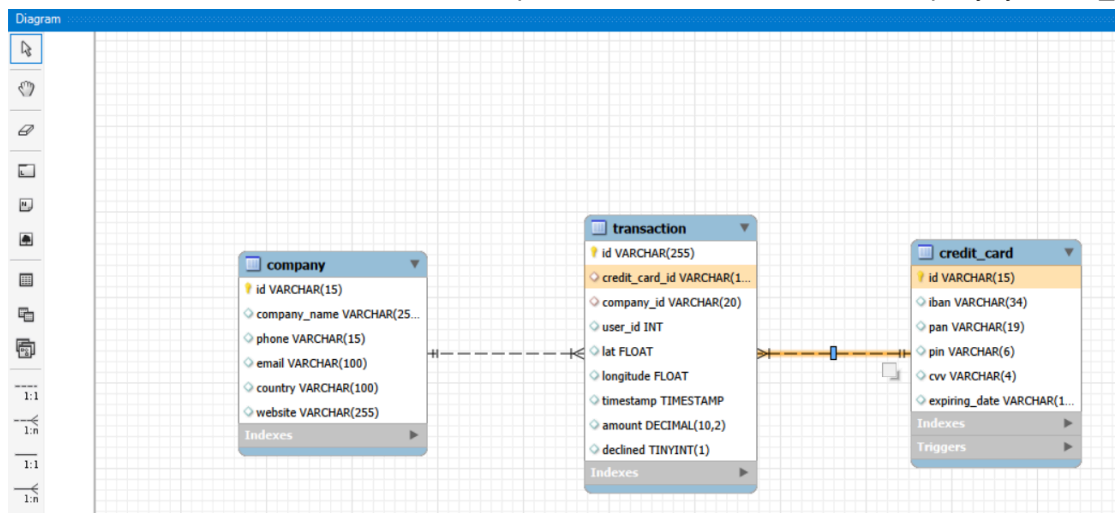
```

#	Time	Action	Message	Duration / Fetch
5010	19:02:26	SHOW CREATE TABLE transaction	1 row(s) returned	0.015 sec / 0.000 sec
5011	19:09:20	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
5012	19:09:24	SELECT * FROM transactions.transaction	100000 row(s) returned	0.000 sec / 0.250 sec
5013	19:11:01	ALTER TABLE transaction ADD CONSTRAINT fk_credit_card FOREIGN KEY (credit_card_id) REFERENCE...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.906 sec

Diagrama

Cardinalidad de 1:n entre las tablas “credit_card” y “transaction”, ya que una tarjeta puede ser usada en múltiples transacciones, pero una transacción tendrá los datos de una tarjeta.

“transaction” actúa como tabla de hechos que conecta ambas entidades “company” y “credit_card”.



Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

1. Check

Enunciati primer llurament espe... Enunciati primer llurament espe... SPRING 2 credit_card

```
148 -- Exercici 5
149 -- El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938.
150 -- La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.
151
152 -- Check:
153 • SELECT * FROM transactions.credit_card
154   WHERE id = "CcU-2938";
155
156 -- Update:
157 • UPDATE transactions.credit_card
158   SET iban = "TR323456312213576817699999"
159   WHERE id = "CcU-2938";
```

Result Grid

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30

credit_card 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5014	19:11:22	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
5015	19:51:02	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
5016	19:51:43	SELECT * FROM transactions.credit_card WHERE id = CcU-2938	Error Code: 1054. Unknown column 'CcU' in 'where clause'	0.000 sec
5017	19:51:52	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
5018	19:55:26	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

2. Update

Enunciati primer llurament espe... Enunciati primer llurament espe... SPRING 2 credit_card

```
140 -- 4. Se establece la relacion de 1:n con la tabla transaction especificando que transaction.credit_card_id es clave foranea y apunta a credit_card.id :
141
142 ALTER TABLE transaction
143 ADD CONSTRAINT fk_credit_card
144 FOREIGN KEY (credit_card_id)
145 REFERENCES credit_card(id);
146
147
148 -- Exercici 5
149 -- El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938.
150 -- La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.
151
152 -- Check:
153 • SELECT * FROM transactions.credit_card
154   WHERE id = "CcU-2938";
155
156 -- Update:
157 • UPDATE transactions.credit_card
158   SET iban = "TR323456312213576817699999"
159   WHERE id = "CcU-2938";
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5015	19:51:02	SELECT * FROM transactions.credit_card	5000 row(s) returned	0.000 sec / 0.000 sec
5016	19:51:43	SELECT * FROM transactions.credit_card WHERE id = CcU-2938	Error Code: 1054. Unknown column 'CcU' in 'where clause'	0.000 sec
5017	19:51:52	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
5018	19:55:26	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
5019	19:57:38	UPDATE transactions.credit_card SET iban = "TR323456312213576817699999" WHERE id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec

3. Confirmación

Enunciat primer lliurament espe... Enunciat primer lliurament espe... SPRINT 2 credit_card

```

152 -- Check:
153 • SELECT * FROM transactions.credit_card
154 WHERE id = "CcU-2938";
155
156 -- Update:
157 • UPDATE transactions.credit_card
158 SET iban = "TR323456312213576817699999"
159 WHERE id = "CcU-2938";
160
161 -- Re check:
162 • SELECT * FROM transactions.credit_card
163 WHERE id = "CcU-2938";

```

Result Grid

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30

credit_card 2 x

Output

#	Time	Action	Message	Duration / Fetch
5016	19:51:43	SELECT * FROM transactions.credit_card WHERE id = CcU-2938	Error Code: 1054. Unknown column 'CcU' in 'where clause'	0.000 sec
5017	19:51:52	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
5018	19:55:26	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
5019	19:57:38	UPDATE transactions.credit_card SET iban = "TR323456312213576817699999" WHERE id = "CcU-2938"	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
5020	19:59:35	SELECT * FROM transactions.credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:

Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id CcU-9999

company_id b-9999

user_id 9999

lat 829.999

longitude -117.999

amount 111.11

declined 0

Dado que los valores que queremos insertar contienen claves foráneas (company.id y credit_card.id), se importan previamente dichos valores en sus tablas de referencia.

SPRINT 2 transaction credit_card company

```

170 user_id 9999
171 lat 829.999
172 longitude -117.999
173 amount 111.11
174 declined 0*/
175
176 • INSERT INTO transactions.company (id) VALUES ('b-9999');
177 • INSERT INTO transactions.credit_card (id) VALUES ('CcU-9999');
178 • INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 2026-07-01 10:26:36, 111.11, 0);
179 -- Check:
180 • SELECT * FROM transactions.transaction
181 WHERE credit_card_id = "CcU-9999";

```

Result Grid

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	2026-07-01 10:26:36	111.11	0

transaction 3 x

Output

#	Time	Action	Message	Duration / Fetch
10	10:26:26	INSERT INTO transactions.credit_card (id) VALUES ('CcU-9999')	1 row(s) affected	0.016 sec
11	10:26:36	INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined)	1 row(s) affected	0.015 sec
12	10:26:46	SELECT * FROM transactions.credit_card WHERE id = "CcU-9999"	1 row(s) returned	0.000 sec / 0.000 sec
13	10:27:03	SELECT * FROM transactions.transaction WHERE id = "CcU-9999"	0 row(s) returned	0.000 sec / 0.000 sec
14	10:27:27	SELECT * FROM transactions.transaction WHERE credit_card_id = "CcU-9999"	1 row(s) returned	0.000 sec / 0.000 sec

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

The screenshot shows a database management tool interface with a SQL editor and a results pane. The SQL editor contains the following queries:

```
177 • INSERT INTO transactions.transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('10881D1D-5B23-A76C-55EF-C568E49A9900', 'Ccl
178 -- Check:
179 • SELECT * FROM transactions.transaction
180 WHERE credit_card_id = "CcU-9999";
181
182
183 -- Exercici 7
184 -- Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.
185
186 • ALTER TABLE transactions.credit_card DROP COLUMN pan;
187 -- Check:
188 • SELECT * FROM transactions.credit_card
```

The results pane shows a table with the following data:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
CcS-4859	XX782693049142353609370	4983	277	2026-11-26
CcS-4860	XX5559930368835304645299	6876	661	2027-07-27
CcS-4861	XX203518287195191627307	5710	398	2026-04-25

The output pane shows the following messages:

#	Time	Action	Message	Duration / Fetch
12	10:26:46	SELECT * FROM transactions.credit_card WHERE id = "CcU-9999"	1 row(s) returned	0.000 sec / 0.000 sec
13	10:27:03	SELECT * FROM transactions.transaction WHERE id = "CcU-9999"	0 row(s) returned	0.000 sec / 0.000 sec
14	10:27:27	SELECT * FROM transactions.transaction WHERE credit_card_id = "CcU-9999"	1 row(s) returned	0.000 sec / 0.000 sec
15	10:36:57	ALTER TABLE transactions.credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
16	10:37:15	SELECT * FROM transactions.credit_card	5001 row(s) returned	0.000 sec / 0.016 sec

Exercici 8

Descarrega els arxius CSV que trobaràs a l'apartat de recursos:

- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

La taula de products.csv l'utilitzarem més endavant.

1. Se crea una base de datos "merchandising"
2. Se crean las tablas "users", "companies", "transactions" y "credit_cards"

Se crea una tabla única "users" en la que se juntan los datos de american_users.csv y european_users.csv, habiendo creado una nueva columna "continent" para conservar la trazabilidad. Los datos provenientes de american_users precisarán "US" en la columna "continent", y "UE" para los european_users.

Debajo, captura de la nomenclatura de la tabla "credit_cards" a modo de ejemplo.

1 • `SELECT * FROM merchandising.credit_cards;`

#	id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date	card_type	card_renewal_flag
1	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

credit_cards 1 x

Output

Action Output

#	Time	Action	Message
✓ 18	16:39:43	CREATE TABLE IF NOT EXISTS users (id VARCHAR(15) PRIMARY KEY, 'name' VARCHAR(255)....	0 row(s) affected
✓ 19	16:39:51	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(15) PRIMARY KEY, company_na...	0 row(s) affected
✓ 20	16:40:02	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(100) PRIMARY KEY, card_id VARCHAR(1...	0 row(s) affected
✓ 21	16:40:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(3...	0 row(s) affected
✓ 22	16:40:39	SELECT * FROM merchandising.credit_cards	0 row(s) returned

3. Se ejecutan los archivos csv correspondientes para insertar los datos en las tablas que acabamos de crear.

```

257 -- En este punto se ejecuta los archivos csv correspondientes para insertar los datos de las tablas que acabamos de crear.
258 -- ejemplo:
259 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
260 INTO TABLE transactions
261 FIELDS TERMINATED BY ';'
262 OPTIONALLY ENCLOSED BY '"'
263 LINES TERMINATED BY '\n'
264 IGNORE 1 ROWS;

```

Output

Action Output

#	Time	Action	Message
✓ 21	16:40:22	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) PRIMARY KEY, user_id VARCHAR(3...	0 row(s) affected
✓ 22	16:40:39	SELECT * FROM merchandising.credit_cards	0 row(s) returned
✓ 23	16:46:11	SELECT * FROM merchandising.users	0 row(s) returned
✓ 24	16:46:27	SELECT * FROM sells.users	5000 row(s) returned
✓ 25	16:51:08	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv' INTO ...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

Tabla "transactions" con los datos cargados a modo de ejemplo.

1 • SELECT * FROM merchandising.transactions;												
Result Grid												
id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount	shipping_u
00043A49-2949-494B-ASDO-ASBAE38B19DD	CS-9294	b-2458	2024-08-28 07:16:46	433.51	0	16, 26, 97, 87	4713	46.19992926158272	1.4355402821327607	34.17	72.25	0.00
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.59720554463741	12.22175994259365	19.92	29.86	6.99
00045D68-ED2E-4F2F-8186-CEE074D875D0	CS-6699	b-2390	2020-07-14 15:37:45	332.38	0	30, 11, 16, 81	2118	29.757295899964348	-95.37963676492454	16.10	22.47	0.00
000481C3-1C26-4FEF-83A0-4CDOEB0048B0	CS-6696	b-2230	2017-09-04 19:44:53	175.00	0	72	2115	53.54888376797025	-113.50305274646564	1.05	14.45	0.00
00051AA4-9CBE-4268-8070-C38062A1B3E2	CS-7606	b-2266	2017-01-05 18:19:25	158.55	0	18	3025	52.20836951654172	5.690806474241335	25.31	25.96	8.99
0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D	CS-3358	b-2598	2023-09-23 04:51:43	317.97	0	35, 33, 19	215	53.553485560256014	-113.4991339874781	2.87	26.25	0.00
0009A151-9BCE-4E31-9053-A468FF77FAAB	CS-7509	b-2546	2023-12-31 00:06:36	457.51	0	93, 55, 28, 91	2928	52.522483604690265	13.552942314307499	5.52	79.40	0.00
0009D494-6245-4DF9-955D-2C084191CFFB	CS-8483	b-2526	2017-07-18 07:52:02	225.49	0	55, 8, 72	3902	45.49200875032752	-73.57063686984957	2.85	17.55	12.99
000A1DEC-CD66-4AB2-A619-71DAB8D4A262	CS-6467	b-2558	2018-09-08 05:29:58	374.41	0	46, 56, 73	1886	55.74247746345254	-3.300093680889247	27.93	62.40	0.00

#	Time	Action	Message	Duration / Fec
22	16:40:39	SELECT * FROM merchandising.credit_cards	0 row(s) returned	0.000 sec / 0.
23	16:46:11	SELECT * FROM merchandising.users	0 row(s) returned	0.016 sec / 0.
24	16:46:27	SELECT * FROM merchandising.transactions	5000 row(s) returned	0.000 sec / 0.
25	16:51:08	LOAD DATA INFILE 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\N1-Ex_8_transactions.csv' INTO ...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	3.516 sec
26	16:52:41	SELECT * FROM merchandising.transactions	100000 row(s) returned	0.000 sec / 0.

4. Se crean las relaciones entre las tablas.

265

-- Se crean las relaciones entre las tablas, de 1:n, donde "1" es transactions, y "n" son las tablas companies, credit_cards y users:

266

• ALTER TABLE merchandising.transactions

268 ADD CONSTRAINT fk_transactions_companies

269 FOREIGN KEY (business_id) REFERENCES companies(company_id),

270 ADD CONSTRAINT fk_transactions_credit_cards

271 FOREIGN KEY (card_id) REFERENCES credit_cards(id),

272 ADD CONSTRAINT fk_transactions_users

273 FOREIGN KEY (user_id) REFERENCES users(id);

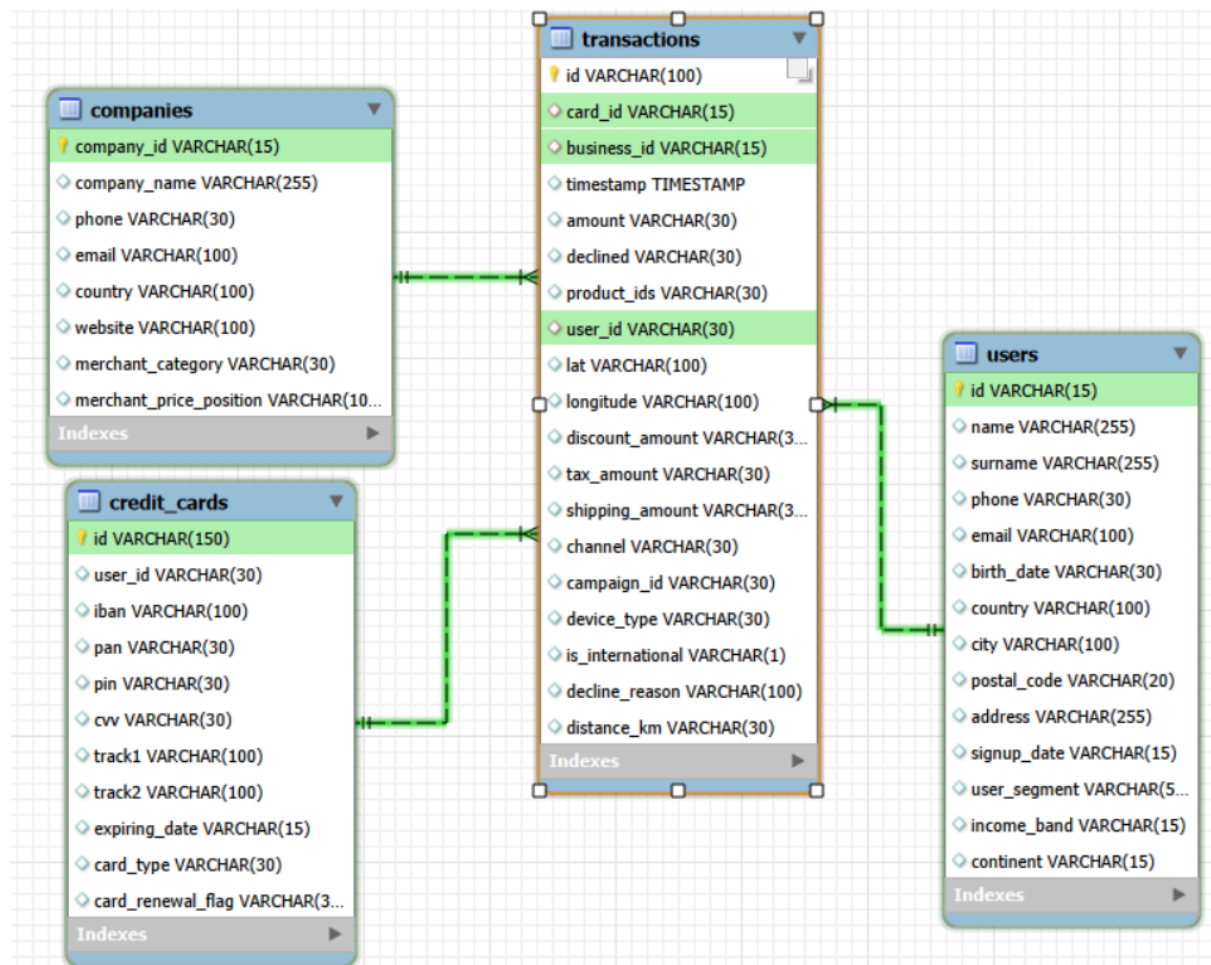
274

Output

📄 Action Output

#	Time	Action	Message
✓ 48	17:20:05	SELECT * FROM merchandising.credit_cards	5000 row(s) returned
✗ 49	17:23:51	LOAD DATA INFILE 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\N1-Ex_8_Users.csv' INTO ...	Error Code: 1406. Data too long for column 'id' at row 1
✓ 50	17:24:32	LOAD DATA INFILE 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\N1-Ex_8_Users.csv' INTO ...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
✓ 51	17:24:36	SELECT * FROM merchandising.transactions	5000 row(s) returned
✗ 52	17:33:33	ALTER TABLE transaction ADD CONSTRAINT fk_transactions_company FOREIGN KEY (company_id) REFE...	Error Code: 1146. Table 'merchandising.transaction' doesn't exist
✓ 53	17:34:22	ALTER TABLE merchandising.transactions ADD CONSTRAINT fk_transactions_company FOREIGN KEY (co...	Error Code: 1072. Key column 'company_id' doesn't exist in table
✓ 54	17:34:38	SELECT * FROM merchandising.transactions	100000 row(s) returned
✗ 55	17:35:26	ALTER TABLE merchandising.transactions ADD CONSTRAINT fk_transactions_company FOREIGN KEY (bu...	Error Code: 1824. Failed to open the referenced table 'company'
✗ 56	17:40:22	ALTER TABLE merchandising.transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (b...	Error Code: 3734. Failed to add the foreign key constraint. Missing column 'id' for constraint 'fk_transactions_co...
✓ 57	17:42:29	SELECT * FROM merchandising.transactions	100000 row(s) returned
✓ 58	17:45:11	ALTER TABLE merchandising.transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (b...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Diagrama



Hemos creado un modelo estrella en el cual la tabla de hechos “transactions” está en el centro y conecta con las tablas de dimensión “users”, “companies” y “credit_cards”.

“transactions” almacena métricas como “amount”, mientras las 3 tablas de dimensión contienen los datos descriptivos de sus atributos respectivos.

La cardinalidad de “transactions” hacia las 3 tablas de dimensión es de 1:N.

Un usuario puede realizar muchas transacciones, una tarjeta puede generar varias transacciones, pero una transacción está realizada por un usuario y generada por una tarjeta.

transactions.card_id es clave foránea
transactions.business_id es clave foránea
transactions.user_id es clave foránea

- apunta a la clave primaria credit_cards.id
- apunta a la clave primaria companies.company_id
- apunta a la clave primaria users.id

Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
--
273 -- Exercici 9
274 -- Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
275
276 • SELECT u.id, u.`name`, u.surname
277 FROM users u
278 WHERE EXISTS (
279     SELECT 1
280     FROM transactions t
281     WHERE t.user_id = u.id
282     AND t.declined = 0
283     GROUP BY t.user_id
284     HAVING COUNT(t.id) > 80
285 );
286
```

Result Grid

	id	name	surname
▶	185	Molly	Gilliam
	289	Dxwgi	Hwrcu
	318	Bnyr	Astuw
	454	Sfzzoh	Xgvfridxs
	NULL	NULL	NULL

users 18 ×

Output

Action Output

#	Time	Action	Message
75	18:41:40	SELECT u.id, u.`name`, u.surname FROM users u WHERE EXISTS (SELECT t.user_id FROM transaccio...	4 row(s) returned
76	18:42:52	SELECT u.id, u.`name`, u.surname FROM users u WHERE EXISTS (SELECT 1 FROM transactions t ...	4 row(s) returned

Exercici 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
288 -- Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
289
290 • SELECT cc.iban, ROUND(AVG(t.amount), 2) AS media_amount
291 FROM transactions t
292 JOIN credit_cards cc ON t.card_id = cc.id
293 JOIN companies c ON t.business_id = c.company_id
294 WHERE c.company_name = 'Donec Ltd'
295 AND t.declined = 0
296 GROUP BY cc.iban;
297
```

Result Grid

	iban	media_amount
▶	CA901355773586771626108990	361.01
	US729212642816955651428791	142.56
	US899450803230933276819251	246.9
	US581097432668325541142297	154.01
	CA441226267105857473570574	250.51
	CA332171962352073015086775	203.95
	US985359628701198880628720	438.16
	US788111972997693675524914	520.37
	CA867886529258577442049898	204.63
	US450950742400388023162002	165.08
	US841589234795722857348646	253.93
	CA707350503853933968563487	290.57

Result 19 ×

Output

Action Output

#	Time	Action	Message
76	18:42:52	SELECT u.id, u.`name`, u.surname FROM users u WHERE EXISTS (SELECT 1 FROM transactions t ...	4 row(s) returned
77	18:47:39	SELECT cc.iban, ROUND(AVG(t.amount), 2) AS media_amount FROM transactions t JOIN credit_cards cc O...	367 row(s) returned

Nivell 2

Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```
299 -- Exercici 1
300 -- Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada tran
301
302 • SELECT DATE(timestamp) AS dia, ROUND(SUM(amount), 2) AS ingresos
303 FROM transactions t
304 WHERE t.declined = 0
305 GROUP BY DATE(timestamp)
306 ORDER BY ingresos DESC
307 LIMIT 5;
308
```

Result Grid

	dia	ingresos
▶	2023-02-20	15728.4
	2019-11-18	15447.89
	2022-12-13	14944
	2019-03-18	14896.47
	2017-12-20	14367.96

Result 21 x

Output

Action Output

#	Time	Action	Message
79	18:53:05	SELECT DATE(timestamp) AS dia, ROUND(SUM(amount), 2) AS ingresos FROM transactions WHERE t.declined = 1	Error Code: 1054. Unknown column 't.declined' in 'where clause'
80	18:53:20	SELECT DATE(timestamp) AS dia, ROUND(SUM(amount), 2) AS ingresos FROM transactions t WHERE t.declined = 1	5 row(s) returned

Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```

309 -- Exercici 2
310 -- Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre
311
312 • SELECT company_name, phone, country, DATE(timestamp) AS fecha, ROUND(amount, 2) AS valor
313 FROM   companies c
314 INNER JOIN transactions t ON c.company_id = t.business_id
315 WHERE  amount BETWEEN 350 AND 400 AND DATE(timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
316 ORDER BY amount DESC;
317

```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	company_name	phone	country	fecha	valor
▶	Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
	Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
	Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.2
	Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
	Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
	Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 23 x

Output

Action Output

#	Time	Action	Message
81	18:55:27	SELECT company_name, phone, country, DATE(timestamp), ROUND(amount, 2) FROM companies c INNER JOIN payments p ON c.id = p.company_id	6 row(s) returned
82	18:56:07	SELECT company_name, phone, country, DATE(timestamp) AS fecha, ROUND(amount, 2) AS valor FROM companies c INNER JOIN payments p ON c.id = p.company_id	6 row(s) returned

Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```
323 • SELECT
324     t.business_id,
325     COUNT(t.id) AS total_transacciones,
326     CASE
327         WHEN COUNT(t.id) >= 400 THEN 'igual_o_mas_de_400'
328         ELSE 'menos_de_400'
329     END AS Transacciones
330 FROM transactions t
331 GROUP BY t.business_id
332 ORDER BY Transacciones DESC;
333
```

business_id	total_transacciones	Transacciones
b-2470	399	menos_de_400
b-2286	397	menos_de_400
b-2362	380	menos_de_400
b-2222	2401	igual_o_mas_de_400
b-2226	410	igual_o_mas_de_400
b-2230	447	igual_o_mas_de_400

Result 24 x

Output

Action Output

#	Time	Action	Message
82	18:56:07	SELECT company_name, phone, country, DATE(timestamp) AS fecha, ROUND(amount, 2) AS valor FROM c...	6 row(s) returned
83	18:58:27	SELECT t.business_id, COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 ...	100 row(s) returned

Exercici 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

1. Se comprueba si existe el registro

```
334 -- Exercici 4
335 -- Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
336
337 -- Check
338 • SELECT * FROM transactions t
339 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
340 -- Delete en la tabla transactions
341 • DELETE FROM transactions t
342 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
343 -- Recheck
344 • SELECT * FROM transactions t
345 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
```

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.59720554463741	12.22175994259365	19.92	29.86

transactions 25 x

Output

Action Output

#	Time	Action	Message
86	19:00:06	SELECT * FROM merchandising.credit_cards	5000 row(s) returned
87	19:00:49	SELECT * FROM transactions t WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) returned

2. Se elimina

```
340 -- Delete en la tabla transactions
341 • DELETE FROM transactions t
342 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
343 -- Recheck
344 • SELECT * FROM transactions t
345 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
346
347
```

Output			
Action Output			
#	Time	Action	Message
✓ 87	19:00:49	SELECT * FROM transactions t WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) returned
✓ 88	19:02:58	DELETE FROM transactions t WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected

3. Se comprueba el borrado

```
343 -- Recheck
344 • SELECT * FROM transactions t
345 WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
346
347
```

Result Grid														
Filter Rows:														
id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount	shipping_amount	channel	campaign
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transactions 26 x

Output

Action Output

#	Time	Action	Message
88	19:02:58	DELETE FROM transactions t WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected
89	19:04:20	SELECT * FROM transactions t WHERE id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	0 row(s) returned

Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

```
353 • CREATE VIEW VistaMarketing AS
354     SELECT company_name, country, phone, ROUND(AVG(t.amount), 2) as media_compras
355     FROM   companies c
356     INNER JOIN transactions t ON c.company_id = t.business_id
357     GROUP BY business_id;
358
359 • SELECT * FROM VistaMarketing
360     ORDER BY media_compras DESC;
```

Result Grid				
Filter Rows:		Export:	Wrap Cell Content:	
company_name	country	phone	media_compras	
Ac Fermentum Incorporated	Germany	+64 601 244 6320	330.77	
Ut Semper Foundation	Sweden	+10 283 500 6449	307.37	
Orci Adipiscing Limited	United Kingdom	+60 897 390 5820	306.27	
Dolor Vitae Limited	France	+6 421 567 9195	305.95	
Nunc In Foundation	Italy	+13 267 741 2619	305.4	
Maecenas Malesuada Fringilla Inc.	Netherlands	+51 673 540 5289	305.15	
Aliquet Sem Limited	Netherlands	+44 744 425 9915	304.21	
Nulla Integer Vulputate Corp.	Sweden	+75 236 687 9162	303.42	
Non Justo Corp.	Sweden	+67 963 294 1666	303.3	
Turpis Company	Netherlands	+17 624 311 7168	302.77	
Aliquam Erat Volutpat LLP	Italy	+39 372 330 7229	301.97	
Amet Lorem LLP	Spain	+40 468 847 7738	301.82	
Placenet LLP	Netherlands	+17 374 607 5033	301.61	

VistaMarketing 29 x

Output

Action Output

#	Time	Action	Message
94	19:15:43	SELECT * FROM merchandising.vistamarketing	100 row(s) returned
95	19:18:21	SELECT * FROM VistaMarketing ORDER BY media_compras DESC	100 row(s) returned

Nivell 3

Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

```

362 -- Nivell 3
363 -- Exercici 1
364 -- Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat deci
365 -- si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon: Quantes targetes estan actives?
366
367 • CREATE TABLE credit_cards_status AS
368 WITH orden_tarjetas AS (
369     SELECT card_id, declined, 'timestamp',
370     ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS orden
371     FROM transactions
372 )
373 SELECT card_id, 'timestamp',
374     IF(declined = 0, 'activa', 'inactiva') AS estado_tarjeta
375 FROM orden_tarjetas
376 WHERE orden IN (1,2,3);

```

Output				
Action Output				
#	Time	Action	Message	
114	20:00:40	WITH orden_tarjetas AS (SELECT card_id, declined, 'timestamp', ROW_NUMBER() OVER ...	15000 row(s) returned	
115	20:04:35	CREATE TABLE credit_cards_status AS WITH orden_tarjetas AS (SELECT card_id, declined, 'timestamp', ...	15000 row(s) affected Records: 15000 Duplicates: 0 Warnings: 0	

Tabla “credit_cards_status” creada

```

1 • SELECT * FROM merchandising.credit_cards_status;

```

Result Grid			
card_id	timestamp	estado_tarjeta	
CcS-4857	2024-10-25 13:11:54	activa	
CcS-4857	2024-10-07 16:43:17	activa	
CcS-4857	2024-07-27 10:50:49	activa	
CcS-4858	2024-08-28 13:48:22	activa	
CcS-4858	2023-12-10 10:56:40	activa	
CcS-4858	2023-12-10 09:21:55	activa	
CcS-4859	2024-06-15 21:04:11	activa	
CcS-4859	2023-11-11 15:18:56	activa	
CcS-4859	2023-03-11 06:20:59	activa	
CcS-4860	2024-10-02 08:31:13	activa	
CcS-4860	2024-06-06 18:23:47	activa	
CcS-4860	2023-07-14 06:59:27	activa	
CcS-4861	2024-07-30 10:30:48	activa	
CcS-4861	2024-07-30 04:15:25	activa	

Output				
Action Output				
#	Time	Action	Message	
1	11:49:21	SELECT * FROM merchandising.credit_cards_status	15000 row(s) returned	

👉 Quantes targetes estan actives?

```
378 -- Partint d'aquesta taula respon: Quantes targetes estan actives?
379
380 • SELECT COUNT(*) AS tarjetas_activas
381 FROM (
382     SELECT card_id
383     FROM credit_cards_status
384     GROUP BY card_id
385     HAVING COUNT(CASE WHEN estado_tarjeta = 'inactiva' THEN 1 END) < 3
386 ) AS activas;
```

Result Grid

tarjetas_activas
5000

Result 30 x

Output

Action Output

#	Time	Action	Message
121	20:30:09	SELECT COUNT(*) AS tarjetas_activas FROM (SELECT card_id FROM credit_cards_status GROUP ...	Error Code: 1248. Even
122	20:30:23	SELECT COUNT(*) AS tarjetas_activas FROM (SELECT card_id FROM credit_cards_status GROUP ...	1 row(s) returned

Exercici 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

1. Se crea la tabla "products"

```
388 -- Exercici 2
389 -- Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades
390 -- tenint en compte que des de transaction tens product_ids. Genera la següent consulta: Necessitem
391
392 -- Creamos la tabla products
393 • CREATE TABLE IF NOT EXISTS products (
394     id VARCHAR(200) PRIMARY KEY,
395     product_name VARCHAR(200),
396     price VARCHAR(15),
397     colour VARCHAR(30),
398     weight VARCHAR(15),
399     warehouse_id VARCHAR(15),
400     category VARCHAR(200),
401     brand VARCHAR(200),
402     cost VARCHAR(30),
403     launch_date VARCHAR(30)
404 );
```

Output

Action Output

#	Time	Action	Message
123	20:32:55	SELECT COUNT(*) AS tarjetas_sin_activas FROM (SELECT card_id FROM credit_cards_status...	1 row(s) returned
124	20:34:37	CREATE TABLE IF NOT EXISTS products (id VARCHAR(200) PRIMARY KEY, product_name VARCHAR...	0 row(s) affected

2. Insertamos los datos desde N1-Ex.8__products.csv.

```

406 -- En este punto se ejecuta el archivo csv correspondiente para insertar los datos en la tabla produ
407 • LOAD DATA INFILE 'C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv'
408 INTO TABLE products
409 FIELDS TERMINATED BY ','
410 OPTIONALLY ENCLOSED BY '"'
411 LINES TERMINATED BY '\\n'
412 IGNORE 1 ROWS;

```

Output			
Action Output			
#	Time	Action	Message
✓ 125	20:35:46	LOAD DATA INFILE 'C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv' INT...	100 row(s) affected Record
✓ 126	20:36:14	SELECT * FROM merchandising.products	100 row(s) returned

Tabla “products” con los datos cargados

1 •	SELECT * FROM merchandising.products;
-----	---------------------------------------

Result Grid										
	id	product_name	price	colour	weight	warehouse_id	category	brand	cost	launch_date
▶	1	Direwolf Stannis	\$161.11	#7c7c7c	1.0	WH-4	Sports	Runway Pro	\$63.84	2014-11-01
	10	Karstark Dorne	\$119.52	#f4f4f4	2.4	WH-5	Food	FreshBox	\$89.07	2013-10-04
	100	south duel	\$40.43	#6d6d6d	3.0	WH-95	Electronics	PixelForge	\$24.95	2013-01-05
	11	Karstark Dorne	\$49.70	#141414	2.7	WH-6	Electronics	Nexa	\$34.45	2014-09-01
	12	duel Direwolf	\$181.60	#a8a8a8	2.1	WH-7	Gaming	Questbyte	\$99.46	2013-10-11
	13	palpatine chewbacca	\$139.59	#2b2b2b	1.0	WH-8	Travel	CityHop	\$82.76	2014-04-26
	14	Direwolf	\$147.53	#c4c4c4	2.0	WH-9	Home	NordHaus	\$71.07	2014-03-29
	15	Stannis warden	\$194.29	#dbdbdb	1.5	WH-10	Home	CasaNova	\$107.93	2014-10-21

products 1 x

Output			
Action Output			
#	Time	Action	Message
✓ 125	20:35:46	LOAD DATA INFILE 'C:\\ProgramData\\MySQL\\MySQL Server 8.0\\Uploads\\N1-Ex.8__products.csv' INT...	100 row(s) affected Record
✓ 126	20:36:14	SELECT * FROM merchandising.products	100 row(s) returned

3. Normalización: En la tabla transactions, los campos de la columna product_ids contienen múltiples valores (ex: “16, 26, 97, 87”), es necesario que las tablas tengan valores atómicos.

Para ello, primero se crea una tabla intermedia entre transactions y products: “transaction_products”.

```

414 -- Se crea una tabla intermedia entre transactions y products
415 CREATE TABLE transaction_products (
416     id INT AUTO_INCREMENT PRIMARY KEY,
417     transaction_id VARCHAR(100) NOT NULL,
418     product_id VARCHAR(100) NOT NULL,
419     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
420     FOREIGN KEY (product_id) REFERENCES products(id)
421 );

```

Output

Action Output

#	Time	Action	Message
126	20:36:14	SELECT * FROM merchandising.products	100 row(s) returned
127	20:40:10	CREATE TABLE transaction_products (id INT AUTO_INCREMENT PRIMARY KEY, transaction_id VAR...	0 row(s) affected

Se emparejan transaction_id con product_id usando JOIN JSON_TABLE. Para ello se convierten primero los product_ids separados por comas en filas (array JSON)

```

424 INSERT INTO transaction_products (transaction_id, product_id)
425 SELECT
426     t.id,
427     CAST(jt.product_id AS UNSIGNED)
428 FROM transactions t
429 JOIN JSON_TABLE(
430     CONCAT('[', t.product_ids, ']'),
431     '$[*]' COLUMNS (
432         product_id VARCHAR(50) PATH '$'
433     )
434 ) AS jt;
435
436 -- Se extrae cuantas veces se ha vendido cada producto
437 SELECT product_id, COUNT(product_id) AS recuento

```

Output

Action Output

#	Time	Action	Message
8	12:26:59	SELECT * FROM merchandising.transaction_products	252569 row(s) returned
9	12:27:37	TRUNCATE TABLE merchandising.transaction_products	0 row(s) affected
10	12:27:42	SELECT * FROM merchandising.transaction_products	0 row(s) returned
11	12:28:27	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, CAST(jt.product_id AS UN...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0

Tabla “transaction_products” con los datos cargados. Ahora, cada fila establece una relación entre un product_id único y su correspondiente transaction_id, y esto nos permite hacer el recuento de venta para cada product_id.

1 • `SELECT * FROM merchandising.transaction_products;`

Result Grid

	id	transaction_id	product_id
▶	1	00043A49-2949-494B-A5DD-A58AE38B19DD	16
	2	00043A49-2949-494B-A5DD-A58AE38B19DD	26
	3	00043A49-2949-494B-A5DD-A58AE38B19DD	97
	4	00043A49-2949-494B-A5DD-A58AE38B19DD	87
	5	00045D68-ED2E-4F2F-8186-CEE074D875D0	30
	6	00045D68-ED2E-4F2F-8186-CEE074D875D0	11
	7	00045D68-ED2E-4F2F-8186-CEE074D875D0	16
	8	00045D68-ED2E-4F2F-8186-CEE074D875D0	81
	9	000481C3-1C26-4FEF-83A0-4CD0EB004BBD	72
	10	00051AA4-9CBE-4268-B070-C38062A1B3E2	18
	11	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	35
	12	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	33
	13	0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	19
	14	0009A151-9BCF-4E31-9053-A468FF77FAAB	93
	15	0009A151-9BCF-4E31-9053-A468FF77FAAB	55
	16	0009A151-9BCF-4E31-9053-A468FF77FAAB	28

ion_products 1 x

Output

Action Output

#	Time	Action	Message
✓ 9	12:27:37	TRUNCATE TABLE merchandising.transaction_products	0 row(s) affected
✓ 10	12:27:42	SELECT * FROM merchandising.transaction_products	0 row(s) returned
✓ 11	12:28:27	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, CAST(t.product_id AS UN...	253388 row(s) affected
✓ 12	12:29:37	SELECT * FROM merchandising.transaction_products	253388 row(s) returned

👉 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

437 `-- Se extrae cuantas veces se ha vendido cada producto`

438 • `SELECT product_id, COUNT(product_id) AS recuento`

439 `FROM transaction_products`

440 `GROUP BY product_id`

441 `ORDER BY COUNT(product_id) DESC;`

Result Grid

	product_id	recuento
▶	52	2650
	29	2629
	66	2596
	87	2591
	48	2588
	33	2587
	88	2581
	73	2581
	68	2580
	34	2577
	4	2574
	61	2573
	31	2572

Result 32 x

Output

Action Output

#	Time	Action	Message
✓ 130	20:49:20	SELECT product_id, COUNT(product_id) FROM sells.transaction_products GROUP BY product_id ORDER B...	100 row(s) returned
✓ 131	20:50:26	SELECT product_id, COUNT(product_id) AS recuento FROM transaction_products GROUP BY product_id O...	100 row(s) returned

